
Algorithm 1 BALL_CONSTRUCT_SYT_INVERSE_RSK(n, d)

Require: n (length), d (minimum Ulam distance)**Ensure:** Exact Ulam ball $B(\text{id}, r)$ and its size

```
1:  $r \leftarrow d - 1$  ▷ ball radius
2:  $m \leftarrow n - r$  ▷ ball condition:  $\text{LIS}(\pi) \geq m$ 
3:  $\Lambda \leftarrow \{\lambda \vdash n : \lambda_1 \geq m\}$  ▷ admissible shapes
4:  $B \leftarrow \emptyset$  ▷ will store  $B(\text{id}, r)$ 
5:  $total \leftarrow 0$ 
6: for all  $\lambda \in \Lambda$  do
7:    $f^\lambda \leftarrow \frac{n!}{\prod_{c \in \lambda} h(c)}$  ▷ hook-length formula
8:    $\mathcal{T}_\lambda \leftarrow \text{ALLSYT}(\lambda)$  ▷ all SYT of shape  $\lambda$ 
9:   for all  $P \in \mathcal{T}_\lambda$  do
10:    for all  $Q \in \mathcal{T}_\lambda$  do
11:       $\pi \leftarrow \text{INVERSE_RSK}(P, Q)$  ▷ perm from two SYT
12:      if  $n - \text{LIS}(\pi) > r$  then
13:        error ▷ sanity check (never triggers)
14:      end if
15:       $B \leftarrow B \cup \{\pi\}$ 
16:       $total \leftarrow total + 1$ 
17:    end for
18:  end for
19: end for
20: return  $(B, total)$ 
```

Algorithm 2 ALLSYT(λ)

Require: Young shape $\lambda \vdash n$ **Ensure:** All standard Young tableaux (SYT) of shape λ

```
1:  $n \leftarrow |\lambda|$ 
2: return GENSYTREC( $\lambda, n$ )
3: procedure GENSYTREC( $\lambda, k$ )
4:   if  $k = 0$  then
5:     return  $\{\emptyset\}$ 
6:   end if
7:    $R \leftarrow \emptyset$ 
8:    $C \leftarrow \text{CORNERS}(\lambda)$  ▷ rows where a corner cell exists
9:   for all  $c \in C$  do
10:     $\lambda' \leftarrow \text{REMOVECORNER}(\lambda, c)$ 
11:     $S \leftarrow \text{GENSYTREC}(\lambda', k - 1)$ 
12:    for all  $T \in S$  do
13:      insert  $k$  into tableau  $T$  at corner  $c$ 
14:       $R \leftarrow R \cup \{T\}$ 
15:    end for
16:  end for
17:  return  $R$ 
18: end procedure
```

Algorithm 3 INVERSE_RSK(P, Q)

Require: SYT P, Q of the same shape $\lambda \vdash n$

Ensure: Permutation $\pi \in S_n$

```
1:  $n \leftarrow$  number of cells of  $Q$ 
2: initialize array  $w[1..n]$ 
3: for  $k \leftarrow n$  down to 1 do
4:   find position  $(r, c)$  of entry  $k$  in  $Q$ 
5:   delete cell  $(r, c)$  from  $Q$ 
6:    $x \leftarrow P[r, c]$ 
7:   delete cell  $(r, c)$  from  $P$ 
8:   for  $i \leftarrow r - 1$  down to 0 do
9:     find rightmost entry  $y < x$  in row  $P[i]$ 
10:    replace  $y$  by  $x$ 
11:     $x \leftarrow y$ 
12:   end for
13:    $w_k \leftarrow x$ 
14: end for
15:  $\pi \leftarrow (w_1, w_2, \dots, w_n)$ 
16: return  $\pi$ 
```

Algorithm-wise explanation (what each algorithm does, then step-by-step details)

This subsection explains Algorithms 1–3 in an *algorithm-wise* manner: for each algorithm, we first state **what it does**, then we give a **fresh, detailed, step-by-step explanation** (A–Z).

Algorithm 1: BALL_CONSTRUCT_SYT_INVERSE_RSK(n, d)

What this algorithm does (one clear statement). It computes the **exact Ulam ball** of radius $r = d - 1$ around the identity permutation:

$$B(\text{id}, r) = \{\pi \in S_n : d_U(\pi, \text{id}) \leq r\},$$

by enumerating *all* permutations in the ball using the Young-tableaux/RSK construction.

Step-by-step explanation (line-by-line meaning).

1. **Set the radius.** The algorithm defines the greedy/GV radius

$$r \leftarrow d - 1.$$

This is the standard radius used for both ball computations and GV-type arguments.

2. **Convert the distance condition into an LIS condition.** For permutations measured against the identity,

$$d_U(\pi, \text{id}) = n - \text{LIS}(\pi).$$

Hence

$$d_U(\pi, \text{id}) \leq r \iff \text{LIS}(\pi) \geq n - r.$$

The algorithm sets

$$m \leftarrow n - r,$$

so that the ball membership condition becomes $\text{LIS}(\pi) \geq m$.

3. **Translate the LIS condition into a Young-shape condition using RSK.** Under the RSK correspondence, each $\pi \in S_n$ maps bijectively to a pair (P, Q) of SYT of the same shape $\lambda \vdash n$. A key property is

$$\text{LIS}(\pi) = \lambda_1,$$

the first-row length of λ . Therefore, $\text{LIS}(\pi) \geq m$ is equivalent to

$$\lambda_1 \geq m.$$

The algorithm forms the set of admissible shapes:

$$\Lambda \leftarrow \{\lambda \vdash n : \lambda_1 \geq m\}.$$

4. **Initialize output containers.** Set $B \leftarrow \emptyset$ to store all permutations in the ball, and $total \leftarrow 0$ to count them.
5. **Process each admissible shape λ .** For each $\lambda \in \Lambda$, the algorithm does three tasks:

- (a) **Compute the tableau count f^λ (hook-length formula).** The number of SYT of shape λ is

$$f^\lambda = \frac{n!}{\prod_{c \in \lambda} h(c)}, \quad h(c) = \#(\text{right}) + \#(\text{below}) + 1.$$

(In the program this value is printed and used as a correctness check.)

- (b) **Generate all SYT of shape λ .** Call Algorithm 2:

$$\mathcal{T}_\lambda \leftarrow \text{ALLSYT}(\lambda).$$

Now $|\mathcal{T}_\lambda| = f^\lambda$.

- (c) **Construct all permutations having shape λ .** RSK implies:

$$\#\{\pi : \text{shape}(\pi) = \lambda\} = (f^\lambda)^2,$$

because each permutation corresponds to an ordered pair (P, Q) of SYT of shape λ . The algorithm iterates over all pairs $(P, Q) \in \mathcal{T}_\lambda \times \mathcal{T}_\lambda$ and constructs the unique permutation:

$$\pi \leftarrow \text{INVERSERSK}(P, Q),$$

by calling Algorithm 3.

Optionally (as in the code), it computes $\text{LIS}(\pi)$ and checks that $n - \text{LIS}(\pi) \leq r$.

Finally, it inserts π into the ball set B and increments the total count.

6. **Return.** After all shapes are processed, the algorithm returns the full list (or set) B and its size $total = |B(\text{id}, r)|$.

Key takeaway. Algorithm 1 computes $B(\text{id}, r)$ *exactly* by reducing the ball condition to a shape filter and then enumerating all pairs of tableaux for each admissible shape.

Algorithm 2: ALLSYT(λ)

What this algorithm does (one clear statement). It generates **all standard Young tableaux (SYT)** of a fixed Young shape λ , using the fundamental fact that the largest label must appear in a corner cell.

Step-by-step explanation (A–Z).

1. **Input is a shape.** The input is a partition $\lambda \vdash n$ (a Young diagram with n cells).
2. **Corner principle.** In any SYT filled with $\{1, 2, \dots, k\}$, the maximum value k must be in a *corner cell* (a cell with no cell to its right and no cell below). If k were not in a corner, then either the row or column strict-increase condition would be violated.
3. **Recursive decomposition.** To list all SYT of shape λ with labels $1..k$:
 - choose a corner cell where k sits,
 - remove that corner cell to get a smaller shape λ' with $k - 1$ cells,
 - recursively list all SYT of shape λ' with labels $1..k - 1$,
 - put k back into the removed corner position.
4. **Enumerate all corners.** The algorithm computes the set $C = \text{CORNERS}(\lambda)$. In shape notation $\lambda = (\lambda_1, \lambda_2, \dots)$, a row index i is a corner row if either i is the last row or $\lambda_i > \lambda_{i+1}$.
5. **Branch on each corner.** For each corner row $c \in C$, remove the last cell of row c to form λ' , solve the subproblem recursively, then insert the value k back in row c .
6. **Base case.** When $k = 0$ (no cells), the only tableau is the empty tableau. This ends recursion.
7. **Output.** The union over all corner branches yields *all* SYT of shape λ , without missing any, because every SYT must place the maximum in *some* corner.

Key takeaway. Algorithm 2 is a complete generator for SYT(λ); it is exact and outputs precisely f^λ tableaux.

Algorithm 3: InverseRSK(P, Q)

What this algorithm does (one clear statement). Given two SYT P and Q of the same shape λ , it computes the **unique permutation** $\pi \in S_n$ corresponding to (P, Q) under RSK, using **reverse row insertion** (*inverse RSK*).

Step-by-step explanation (very detailed).

1. **Why we delete $k = n$ down to 1.** In the forward RSK process, Q records *when* each cell was created: label k is written in the cell created during insertion of the k -th symbol. Therefore, to undo the process, we remove labels in reverse order $k = n, n - 1, \dots, 1$.
2. **Locate the cell of k in Q .** Because Q is an SYT, the largest remaining label k always lies in a corner cell. The algorithm finds its coordinates (r, c) .

3. **Delete that cell from Q .** This removes the “time stamp” k and reduces the shape by one corner cell.
4. **Delete the same cell from P and take its value.** Remove the cell at the same coordinates (r, c) in P . Let the removed entry be x . Intuitively, x is the value that must be pushed upward to recover the symbol w_k .
5. **Reverse bump (reverse row insertion) upward.** Now we move x upward row by row, from row $r - 1$ down to row 0:
 - In the current row, find the *rightmost* entry strictly smaller than x .
 - Swap that entry with x (place x into the row).
 - Update x to be the swapped-out value (this is what continues upward).

This exactly reverses the forward insertion rule (which bumps using the leftmost $> x$).

6. **Output w_k .** After processing up to the top row, the final value of x is assigned to w_k . This is the recovered k -th symbol of the permutation.
7. **Repeat until $k = 1$.** Continuing for $k = n, n - 1, \dots, 1$ recovers the entire permutation

$$\pi = (w_1, w_2, \dots, w_n).$$

8. **Correctness intuition (why it works).** Each deletion reduces the shape by one corner cell in both P and Q . The reverse bumping maintains the increasing row property of P and exactly undoes the earlier forward bump sequence that created that cell. Hence the procedure is a true inverse.

Key takeaway. Algorithm 3 is the exact inverse of RSK on permutations: each pair (P, Q) yields one and only one permutation π .

Final combined conclusion (why this equals the full Ulam ball)

What we get after running all three algorithms together. Algorithm 1:

- keeps only shapes with $\lambda_1 \geq m = n - r$ (equivalently $\text{LIS}(\pi) \geq m$),
- for each such shape λ , enumerates all pairs $(P, Q) \in \text{SYT}(\lambda)^2$,
- converts each (P, Q) to a permutation π by Algorithm 3,
- therefore lists *all* permutations π with $d_U(\pi, \text{id}) \leq r$.

Hence the output is exactly $B(\text{id}, r)$ and

$$|B(\text{id}, r)| = \sum_{\lambda \vdash n, \lambda_1 \geq n-r} (f^\lambda)^2.$$